

ChatSocket Python

MANUAL DO USUÁRIO E DOCUMENTAÇÃO TÉCNICA DE ENGENHARIA

EQUIPE DE DESENVOLVIMENTO / INTEGRANTES

Vinicius de Brito | Alex Garcia | Matheus Farias | Felipe Pereira

1. Introdução e Diretrizes do Projeto

Este documento estabelece o manual oficial de operação e a especificação técnica do sistema **ChatSocket Python**. A aplicação foi desenvolvida sob o modelo de arquitetura distribuída Cliente-Servidor para viabilizar a troca síncrona de mensagens de texto em tempo real entre processos distintos, utilizando exclusivamente os recursos nativos de comunicação em rede da linguagem Python.

O escopo principal do projeto consiste em consolidar, de forma prática, os fundamentos de redes de computadores na camada de transporte do modelo de referência da internet. Através de canais de comunicação de baixo nível, o software abstrai a complexidade do tráfego de pacotes, entregando uma interface simplificada e robusta via linha de comando (CLI) focada na estabilidade e na confiabilidade da transmissão de dados.

2. Fundamentos Teóricos de Rede

2.1 Arquitetura Cliente-Servidor

O sistema opera sob uma topologia em estrela baseada em papéis bem definidos para cada processo:

- **Servidor:** Atua como o nó passivo da rede. Ele realiza a vinculação a uma porta de rede lógica e permanece em estado de escuta bloqueante, aguardando que solicitações externas de conexão sejam instanciadas.
- **Cliente:** Atua como o nó ativo. Ele detém a responsabilidade de conhecer o endereço de rede do servidor e iniciar o processo de conexão (handshake), abrindo ativamente o canal bidirecional.

2.2 Abstração por Sockets TCP

Um *socket* representa o ponto final (endpoint) de um elo de comunicação bidirecional entre dois programas que cooperam através da rede. No ecossistema Python, o sistema utiliza a biblioteca nativa `socket` configurada com parâmetros específicos:

- **AF_INET** : Configura o emprego da família de protocolos IPv4 para o endereçamento de rede.
- **SOCK_STREAM** : Define explicitamente a utilização do protocolo de transporte **TCP (Transmission Control Protocol)**.

A escolha do TCP em detrimento do UDP é mandatória para este cenário. O TCP provê uma comunicação rigidamente orientada à conexão, caracterizada pela garantia absoluta de entrega dos dados na mesma ordem sequencial em que foram transmitidos, mecanismos automáticos de retransmissão em caso de perda de pacotes, controle de fluxo e verificação ativa de erros.

2.3 Parâmetros de Endereçamento e Portas

Para fins de identificação, o sistema foi parametrizado sob os seguintes padrões técnicos:

- **Endereço de Loopback (127.0.0.1 / localhost)**: Utilizado pelo cliente para direcionar o tráfego de pacotes para a própria máquina local, permitindo testes locais de comunicação entre processos e isolamento de rede.
- **Endereço Curinga (0.0.0.0)**: Utilizado no processo de associação do servidor (*bind*). Instruiu o sistema operacional a aceitar requisições originadas em qualquer interface física ou virtual de rede ativa no computador host.
- **Porta Lógica (12345)**: Canal lógico de comunicação fora do intervalo reservado do sistema (portas de 0 a 1023), mitigando conflitos operacionais com serviços padrão como HTTP (80) ou HTTPS (443).

3. Arquitetura e Engenharia de Fluxo

A fim de garantir eficiência e legibilidade, o código foi reestruturado para eliminar a manipulação rudimentar de bytes criptográficos brutos, sendo substituído por um modelo elegante de gerenciamento de buffers e arquivos virtuais através da função nativa `makefile()`.

3.1 Fluxo de Buffers e Canais de Stream

Ao consolidar o acoplamento da rede, o socket bruto é mapeado em dois fluxos de texto independentes: um canal de leitura com permissão de entrada (modo `'r'`) e um canal de escrita com permissão de saída (modo `'w'`). Esse isolamento permite tratar a rede de maneira análoga à leitura e gravação de arquivos de texto comuns do sistema operacional.

Importância do Mecanismo Flush

Como os streams de texto operam de forma nativa com buffers temporários em memória para otimizar a performance de escrita, o sistema utiliza obrigatoriamente a operação de descarga forçada `flush()` após cada mensagem postada. Sem esta rotina, as strings enviadas ficariam retidas no buffer local do remetente, impossibilitando a exibição em tempo real no terminal do destinatário.

3.2 Sincronismo e Alternância de Turnos

A conversação ocorre através de um ciclo em laço estrito de alternância de turnos, projetado especificamente para interfaces de console. O cliente assume a palavra inicial, enviando o primeiro pacote e entrando em estado de bloqueio. O servidor permanece escutando o buffer até que um caractere de quebra de linha (`\n`) seja detectado pela rotina `readline()` , liberando o prompt para que o operador do servidor possa digitar sua respectiva tréplica. Este encadeamento evita colisões de renderização gráfica nos prompts de comando dos terminais.

4. Manual do Usuário e Operação do Sistema

Esta seção detalha o roteiro prático e obrigatório para compilação, infraestrutura de execução e interação operacional com os módulos do software.

4.1 Pré-requisitos do Sistema

- Interpretador oficial Python instalado (versão 3.8 ou superior recomendada).
- Acesso a duas janelas independentes de prompt de comando (Terminal, Prompt do Windows, PowerShell ou Bash).
- Estrutura de diretório contendo os arquivos originais segmentados: `Servidor.py` e `Cliente.py` .

4.2 Passo a Passo para Execução Sequencial

Para o correto funcionamento do ecossistema, os passos abaixo devem ser seguidos rigorosamente na ordem descrita:

- 1. Inicialização do Servidor (Terminal 1):** Abra o primeiro terminal e navegue até a pasta do projeto. Execute o comando de inicialização da escuta passiva:
`python Servidor.py`
O terminal exibirá a mensagem de conformidade: **"Servidor iniciado. Aguardando conexão na porta 12345..."**. Neste momento, o processo está bloqueado na chamada de sistema aguardando o cliente.
- 2. Inicialização do Cliente (Terminal 2):** Abra o segundo terminal na mesma máquina. Inicialize o processo ativo com o comando:

python Cliente.py

Imediatamente, o cliente efetuará a chamada de conexão ao endereço IP local.

3. **Consolidação do Enlace:** Se a porta estiver liberada, o handshake se completará. O Terminal 1 (Servidor) atualizará seu estado para "**Cliente conectado com sucesso!**" e o Terminal 2 (Cliente) confirmará "**Conectado ao servidor no endereço 127.0.0.1**".
4. **Dinâmica de Conversação:** O operador do Cliente digita a mensagem inicial e aperta a tecla *Enter*. O Servidor recebe o texto em tela e tem seu prompt liberado sob a legenda "Sua resposta:". O diálogo continua de forma alternada.

Regra Crítica de Sequenciamento

Se o script do Cliente for inicializado sem que o processo do Servidor esteja ativamente rodando na porta correspondente, o sistema operacional abortará o programa instantaneamente, emitindo um erro de conexão recusada (`ConnectionRefusedError`). Sempre inicie o servidor primeiro.

4.3 Guia de Comandos e Interações Operacionais

AÇÃO OPERACIONAL	ENTRADA DO USUÁRIO	COMPORTAMENTO E RESPOSTA INTERNA DO SISTEMA
Transmissão de Mensagem	Qualquer string alfanumérica + <i>Enter</i>	O texto é empacotado, recebe terminação de quebra de linha, passa por limpeza de espaçamentos em branco residuais (<code>strip()</code>) e é impresso no terminal oposto.
Encerramento Voluntário	SAIR (independente de maiúsculas)	O loop de repetição local é quebrado imediatamente. O programa envia a palavra-chave de sinalização ao parceiro e fecha de forma segura todos os arquivos e descritores de sockets associados à sessão.
Encerramento por Queda Remota	Nenhuma entrada (Detectada automaticamente)	Se o parceiro remoto fechar a janela do terminal ou digitar o comando de encerramento, o canal lerá uma linha vazia. O sistema intercepta o evento, sinaliza em tela a desconexão do outro lado e encerra a própria execução sem travamentos.

5. Robustez, Exceções e Estabilidade

A estabilidade da aplicação em ambientes reais de rede é garantida por meio de um ecossistema de captura preventiva de exceções encapsulado em blocos lógicos estruturados (`try-except`). Em vez de permitir o encerramento catastrófico do programa com mensagens de rastreamento de pilha confusas (stack traces), o software prevê e intercepta os seguintes problemas:

- **Portas Lógicas Ocupadas (`OSError`):** Ocorre se outra aplicação no sistema operacional já estiver consumindo a porta 12345 no momento em que o servidor tenta realizar o `bind()` . O sistema emite um alerta amigável de erro informando a indisponibilidade do canal lógico.
- **Quedas de Enlace e Canal Nulo:** Caso o cabo de rede seja desconectado ou o processo oposto sofra uma interrupção forçada do sistema operacional (como um encerramento pelo Gerenciador de Tarefas), a leitura do buffer retornará nula. O software captura esse sinal de forma graciosa, encerra os descritores internos abertos e preserva a integridade física da máquina local.

6. Considerações de Evolução Técnica

Embora o ChatSocket cumpra integralmente os objetivos acadêmicos e técnicos propostos de conectividade ponto a ponto síncrona, a arquitetura de engenharia de software prevê os seguintes vetores de escalabilidade futura:

1. **Escalabilidade para Múltiplos Clientes:** Integração de módulos de concorrência paralela por meio de multi-threading (módulo `threading`) ou laços assíncronos não bloqueantes (módulo `asyncio`). Isso permitirá que o servidor gerencie múltiplos sockets simultaneamente sem travar a execução na chamada de um único cliente.
2. **Camada de Segurança de Dados:** Acoplamento de uma camada de transporte segura através de criptografia assimétrica ponta a ponta (empregando o protocolo SSL/TLS nativo do Python), impedindo ataques de interceptação de pacotes (packet sniffing) no meio de transmissão.
3. **Interface Avançada e Persistência:** Substituição da interface CLI por bibliotecas gráficas nativas e implementação de persistência de histórico de conversas através de bancos de dados locais estruturados (como o SQLite).

7. Conclusão

O desenvolvimento e consolidação do projeto **ChatSocket Python** demonstram empiricamente a viabilidade e a simplicidade de implementação de protocolos de comunicação confiáveis na camada de aplicação. Ao mapear conexões primitivas em fluxos de texto gerenciados e assegurar um ciclo de vida robusto, o projeto alcança o patamar de uma ferramenta de aprendizado prática de engenharia de

redes, servindo de fundação estável para sistemas distribuídos e arquiteturas de microserviços contemporâneas.